To-do List

1. task.dart: Modelo de datos

```
class Task {
 final String id;
 final String title;
 final String category;
 bool isCompleted;

 Task({
 required this.id,
 required this.title,
 required this.category,
 this.isCompleted = false,
 });
}
```

Qué hace

- Define la estructura de una tarea.

Es el modelo de dominio (para desarrollo tipo MVC / MVVM):

- Representa el dato puro
- No tiene lógica de UI
- No depende de Flutter

2. task_provider.dart: Gestión de datos

```
class TaskProvider extends ChangeNotifier{
  final List<Task> _tasks = [];
  List<Task> get tasks => _tasks;

  void addTask(String title, String category){
    final newTask = Task(id: DateTime.now().toString(), title: title, category:
category);
    _tasks.add(newTask);
    notifyListeners();
  }

  void toggleTaskStatus(String id){
    final taskIndex = _tasks.indexWhere((task) => task.id == id);
    if (taskIndex!=-1){
      _tasks[taskIndex].isCompleted = !_tasks[taskIndex].isCompleted;
      notifyListeners();
    }
  }
}
```

Qué hace

- Mantiene el **estado global de la app**
- Expone datos (tasks)
- Expone operaciones (addTask, toggleTaskStatus)
- Notifica cambios (notifyListeners())

Cuando se ejecuta:

- Todos los widgets que escuchan este provider se reconstruyen

Por qué usar Provider

Evita:

- pasar datos manualmente entre widgets
- estados duplicados
- lógica en la UI

Permite:

- estado centralizado
- UI reactiva
- desacoplamiento

3. task_item.dart: widget + acción

```
class TaskItem extends StatelessWidget{
  final Task task;
  TaskItem({super.key, required this.task});

  @override
  Widget build(BuildContext context){
    final taskProvider = Provider.of<TaskProvider>(context, listen:false);
    return Card(
      child: ListTile(
        title: Text(
           task.title,
           style: TextStyle(
            decoration: task.isCompleted ? TextDecoration.lineThrough: null
           ), // textstyle
        ), // text

        trailing: Checkbox(
          value: task.isCompleted,
          onChanged: (value){
            taskProvider.toggleTaskStatus(task.id);
          },
        ), // checkbox

      ), // listtile
    );   // card
  }
}
```

Qué hace

- Representa **una tarea individual en la UI**.

Flujo

1. Usuario hace click en checkbox
2. Llama a:
3. toggleTaskStatus(task.id)
4. Provider cambia estado
5. Widget padre se reconstruye
6. Este widget recibe nuevo task

4. task_list_screen.dart: screen

Qué hace

- Obtiene listas de tareas desde Provider
- Renderiza lista

5. main.dart: punto de entrada